

gnuradio_decode_file.py DSP Mathematics and Implementation Reference

August 11, 2026

Abstract

This document explains the digital signal processing calculations implemented by RetroSpectrum's GNU Radio file-demodulation helper. For every major calculation, it shows the relevant Python source, expresses the same operation mathematically, defines the variables, and explains why the calculation is performed. The emphasis is the active decode path: complex16 IQ normalization, fixed samples-per-symbol averaging, power/envelope extraction, OOK/ASK threshold slicing, robust RAW pulse decoding, rectangular moving-average filtering, phase-discriminator demodulation, 2-FSK/GFSK/AFSK slicing, 1-D k-means clustering for 4-FSK, ideal PSK/QAM constellation construction, Euclidean nearest-constellation decisions, and final symbol-to-bit conversion.

Contents

1	Scope and Signal Model	4
1.1	Supported modulation modes	4
2	Complex16 File Loading and IQ Normalization	4
2.1	Byte offset for a requested IQ sample	4
2.2	Signed integer to complex floating-point conversion	5
3	Fixed Samples-per-Symbol Views	5
3.1	Trimming to complete symbol blocks	5
3.2	Complex symbol averaging	6
3.3	Mean symbol power	6
4	Normalization Operations	6
4.1	Complex RMS normalization	6
4.2	Real min-max normalization	7
5	OOK/ASK Symbol-Power Slicer	7
5.1	Midrange threshold	7
6	RAW OOK Pulse Decoding	8
6.1	Envelope power	8
6.2	Rectangular moving-average filter	8
6.3	Percentile-based robust thresholding	9
6.4	Run-length encoding	9
6.5	Merging short threshold fragments	10
6.6	PWM high/low pair decision	10
6.7	Slot-quantized fallback	11

7	Complex Phase Discriminator	11
7.1	Differential phase calculation	11
7.2	Relationship to instantaneous frequency	12
8	Symbol Averaging of Real DSP Features	12
9	2-FSK, GFSK, and AFSK Path	12
9.1	2-FSK median slicing	12
9.2	Additional GFSK smoothing	13
9.3	AFSK dispatch behavior	13
10	One-Dimensional K-Means Clustering	13
10.1	Cluster initialization and assignment	13
11	4-FSK Demodulation	14
12	Ideal PSK and QAM Constellations	14
12.1	Constellation definitions	14
12.2	Constellation power normalization	15
13	PSK/QAM Constellation Demodulation	16
13.1	Optional received-symbol normalization	16
13.2	GNU Radio BPSK/QPSK path	16
13.3	Euclidean nearest-neighbor fallback	16
14	Symbol Label to Bitstream Conversion	17
14.1	Fixed-width MSB-first extraction	17
15	Top-Level Decode Dispatch	17
16	End-to-End Mathematical Paths	18
16.1	OOK symbol slicer	18
16.2	2-FSK / GFSK / AFSK	18
16.3	4-FSK	19
16.4	PSK/QAM	19
17	Worked DSP Examples	19
17.1	OOK symbol-power example	19
17.2	RAW PWM example	19
17.3	2-FSK phase-discriminator example	20
17.4	QPSK nearest-point example	20
18	Engineering Interpretation and Assumptions	20
18.1	Fixed timing is an input assumption	20
18.2	No explicit carrier recovery in the constellation fallback	20
18.3	Differential phase naturally suppresses amplitude scale	21
18.4	Median FSK threshold adapts to common offset	21
18.5	K-means estimates tone levels from the recording	21
18.6	Constellation labeling is implementation-defined by point order	21
19	DSP Function Reference Summary	21

20 Core DSP Source Listings	22
20.1 Complex16 normalization and symbol statistics	22
20.2 Normalization	22
20.3 RAW OOK DSP	22
20.4 FSK-family DSP	23
20.5 4-FSK clustering	23
20.6 PSK/QAM decisions	23

1 Scope and Signal Model

The helper reads interleaved signed 16-bit IQ samples from a RetroSpectrum recording and converts each stored pair into a normalized complex sample

$$x[n] = I_f[n] + jQ_f[n], \quad (1)$$

where

$$I_f[n] = \frac{I_{16}[n]}{32768}, \quad Q_f[n] = \frac{Q_{16}[n]}{32768}. \quad (2)$$

The script does not perform automatic timing recovery. Instead, the caller supplies a samples-per-symbol value

$$N_s = \text{sps}, \quad (3)$$

and the helper groups or averages samples in fixed blocks of that length. The active processing paths are

complex16 IQ $\rightarrow$ fixed symbol blocks $\rightarrow$ OOK/PSK/QAM decisions,
 complex16 IQ $\rightarrow$ envelope $\rightarrow$ RAW OOK run lengths,
 complex16 IQ $\rightarrow$ phase difference $\rightarrow$ 2-FSK/GFSK/AFSK or 4-FSK decisions.

1.1 Supported modulation modes

Relevant code (gnuradio_decode_file.py lines 37–49):

```

37 MODES = [
38     "ook",
39     "ook_raw",
40     "bpsk",
41     "qpsk",
42     "psk8",
43     "fsk2",
44     "gfsk",
45     "fsk4",
46     "afsk",
47     "qam16",
48     "qam64",
49 ]

```

The modes share lower-level DSP primitives instead of each implementing a completely independent signal chain. In particular, FSK-family modes share the phase discriminator, PSK/QAM modes share symbol averaging and constellation decisions, and both OOK modes operate on signal power.

2 Complex16 File Loading and IQ Normalization

2.1 Byte offset for a requested IQ sample

Relevant code (gnuradio_decode_file.py lines 52–76):

```

52 def read_complex16_raw(path: Path, start_sample: int, count_iq: int) -> np.ndarray:
53     if start_sample < 0:
54         raise ValueError("start-sample must be >= 0")
55     if count_iq <= 0:
56         raise ValueError("sample count must be > 0")
57     with path.open("rb") as fp:
58         fp.seek(start_sample * 4, 0)
59         raw = np.fromfile(fp, dtype=np.int16, count=count_iq * 2)
60     if raw.size % 2:
61         raw = raw[:-1]
62     iq = raw.reshape((-1, 2)).astype(np.float32)
63     return ((iq[:, 0] + 1j * iq[:, 1]) / 32768.0).astype(np.complex64)

```

One complex16 IQ sample contains two signed 16-bit integers:

$$\underbrace{I_{16}}_{2 \text{ bytes}} \quad \underbrace{Q_{16}}_{2 \text{ bytes}}. \quad (4)$$

Therefore each IQ sample occupies four bytes, and the byte offset for IQ index n_0 is

$$B_{\text{offset}} = 4n_0. \quad (5)$$

The file read requests twice as many scalar integers as complex samples because each complex sample contains an I and a Q component:

$$N_{\text{int16}} = 2N_{\text{IQ}}. \quad (6)$$

If an odd scalar count is returned, the unmatched final scalar is removed so the data can still be reshaped into complete (I, Q) pairs.

2.2 Signed integer to complex floating-point conversion

The final conversion is

$$x[n] = \frac{I_{16}[n] + jQ_{16}[n]}{32768}. \quad (7)$$

Because a signed 16-bit integer has the range

$$-32768 \leq I_{16}, Q_{16} \leq 32767, \quad (8)$$

the normalized components lie approximately in

$$-1 \leq I_f, Q_f < 1. \quad (9)$$

The divisor is 32768, rather than 32767, so the most-negative representable value maps exactly to -1.0 . The positive maximum maps to $32767/32768 \approx 0.9999695$.

3 Fixed Samples-per-Symbol Views

3.1 Trimming to complete symbol blocks

Relevant code (gnuradio_decode_file.py lines 79–106):

```

79 def symbol_views(path: Path, start_sample: int, max_symbols: int, sps: int):
80     if sps <= 0:
81         raise ValueError("samples/symbol must be > 0")
82     if max_symbols <= 0:
83         raise ValueError("max-symbols must be > 0")
84     c = read_complex16_raw(path, start_sample, max_symbols * sps)
85     usable = (c.size // sps) * sps
86     c = c[:usable]
87     blocks_view = c.reshape((-1, sps))
88     complex_symbols = blocks_view.mean(axis=1).astype(np.complex64)
89     mag_symbols = np.mean(np.abs(blocks_view) ** 2, axis=1).astype(np.float32)
90     return complex_symbols, mag_symbols, c

```

If N IQ samples are available and the supplied samples-per-symbol value is N_s , the usable count is

$$N_u = \left\lfloor \frac{N}{N_s} \right\rfloor N_s. \quad (10)$$

This discards any incomplete tail symbol. The trimmed sequence is then reshaped into a matrix

$$X[k, m] = x[kN_s + m], \quad (11)$$

where k is symbol index and $m \in \{0, \dots, N_s - 1\}$ is sample position inside the symbol.

3.2 Complex symbol averaging

The complex representative for symbol k is

$$\bar{x}_k = \frac{1}{N_s} \sum_{m=0}^{N_s-1} X[k, m]. \quad (12)$$

This is a rectangular integrate-and-dump operation. It reduces random sample noise when the symbol is approximately constant over the supplied timing interval. It also preserves the average phase and amplitude required by the PSK/QAM decision path.

3.3 Mean symbol power

For OOK/ASK, the code separately calculates

$$P_k = \frac{1}{N_s} \sum_{m=0}^{N_s-1} |X[k, m]|^2. \quad (13)$$

Since

$$|x|^2 = I^2 + Q^2, \quad (14)$$

P_k is the average complex-envelope power over symbol k . This removes carrier phase from the OOK decision variable.

4 Normalization Operations

4.1 Complex RMS normalization

Relevant code (gnuradio_decode_file.py lines 109–119):

```

109 def normalize_complex(x: np.ndarray) -> np.ndarray:
114     if x.size == 0:
115         return x.astype(np.complex64)
116     p = float(np.mean(np.abs(x) ** 2))
117     if p <= 0.0 or not math.isfinite(p):
118         return x.astype(np.complex64)
119     return (x / math.sqrt(p)).astype(np.complex64)

```

The estimated average power is

$$P = \frac{1}{N} \sum_{n=0}^{N-1} |x[n]|^2. \quad (15)$$

The RMS complex magnitude is $\sqrt{P}$, so the normalized samples are

$$x_N[n] = \frac{x[n]}{\sqrt{P}}. \quad (16)$$

Then, for valid nonzero P ,

$$\frac{1}{N} \sum_n |x_N[n]|^2 = 1. \quad (17)$$

This removes overall amplitude scale while preserving every sample's phase and relative amplitude.

4.2 Real min-max normalization

Relevant code (gnuradio_decode_file.py lines 122–133):

```

122 def normalize_float(x: np.ndarray) -> np.ndarray:
127     if x.size == 0:
128         return x.astype(np.float32)
129     lo = float(np.min(x))
130     hi = float(np.max(x))
131     if not math.isfinite(lo) or not math.isfinite(hi) or hi <= lo:
132         return x.astype(np.float32)
133     return ((x - lo) / (hi - lo)).astype(np.float32)

```

For a real-valued sequence with minimum $x_{\min}$ and maximum $x_{\max}$, the code applies

$$y[n] = \frac{x[n] - x_{\min}}{x_{\max} - x_{\min}}. \quad (18)$$

Thus

$$\min y = 0, \quad \max y = 1. \quad (19)$$

This is used by the OOK paths so thresholds can operate independently of absolute received power when normalization is enabled.

5 OOK/ASK Symbol-Power Slicer

5.1 Midrange threshold

Relevant code (gnuradio_decode_file.py lines 136–165):

```

136 def run_ook_symbol_slicer(mag_symbols: np.ndarray, normalize: bool) -> list[int]:
146     x = normalize_float(mag_symbols) if normalize else mag_symbols.astype(np.float32)
147     lo = float(np.min(x))
148     hi = float(np.max(x))
149     if hi <= lo:
150         return []
151     threshold = lo + 0.5 * (hi - lo)
152     tb = gr.top_block()
153     src = blocks.vector_source_f(x.tolist(), False)
154     thr = blocks.threshold_ff(threshold, threshold, 0)
155     f2b = blocks.float_to_uchar()
156     sink = blocks.vector_sink_b()
157     tb.connect(src, thr)
158     tb.connect(thr, f2b)
159     tb.connect(f2b, sink)
160     tb.run()
161     return [int(v) & 1 for v in sink.data()]

```

The threshold is the midpoint between the observed low and high values:

$$T = x_{\min} + \frac{1}{2}(x_{\max} - x_{\min}) \quad (20)$$

$$= \frac{x_{\min} + x_{\max}}{2}. \quad (21)$$

The intended binary decision is

$$b_k = \begin{cases} 1, & P_k \geq T, \\ 0, & P_k < T. \end{cases} \quad (22)$$

When min-max normalization is enabled, $x_{\min} = 0$ and $x_{\max} = 1$, so the threshold becomes exactly

$$T = 0.5. \quad (23)$$

The GNU Radio `threshold_ff` block performs the threshold operation, `float_to_uchar` converts its output to bytes, and the final bit mask `& 1` constrains each recovered value to one bit.

6 RAW OOK Pulse Decoding

The RAW OOK path does not assume that one supplied block directly corresponds to one binary bit. Instead, it detects high/low envelope runs and interprets relative pulse widths.

6.1 Envelope power

Relevant code (gnuradio_decode_file.py lines 275–306):

```

275 def run_ook_raw_pulse_decoder(samples: np.ndarray, sps: int, normalize: bool) -> list[int]:
294     if samples.size < 2 or sps <= 0:
295         return []
297     env = (np.abs(samples) ** 2).astype(np.float32)
298     smooth_len = max(1, min(max(1, sps // 20), 128))
299     env_smooth = moving_average(env, smooth_len)
300     env_smooth = normalize_float(env_smooth) if normalize else env_smooth
303     states = threshold_binary_robust(env_smooth)
304     runs = build_runs_from_states(states)
305     runs = merge_short_runs(runs, max(1, sps // 10))
306     bits = decode_pwm_pairs_from_runs(runs, max(1, sps // 5))

```

The envelope variable is actually instantaneous complex-envelope power:

$$E[n] = |x[n]|^2 = I[n]^2 + Q[n]^2. \quad (24)$$

The smoothing length is

$$N_{\text{smooth}} = \max \left(1, \min \left(\max \left(1, \left\lfloor \frac{N_s}{20} \right\rfloor \right), 128 \right) \right). \quad (25)$$

For normal positive N_s , this is approximately $N_s/20$ samples, capped at 128. It suppresses very short envelope spikes while keeping the smoothing window much shorter than the caller's base timing unit.

6.2 Rectangular moving-average filter

Relevant code (gnuradio_decode_file.py lines 339–348):

```

339 def moving_average(x: np.ndarray, n: int) -> np.ndarray:
344     if x.size == 0 or n <= 1:
345         return x.astype(np.float32)
346     n = min(n, x.size)
347     kernel = np.ones(n, dtype=np.float32) / float(n)
348     return np.convolve(x, kernel, mode="same").astype(np.float32)

```

The impulse response of this FIR smoother is

$$h[m] = \begin{cases} \frac{1}{N}, & 0 \leq m < N, \\ 0, & \text{otherwise.} \end{cases} \quad (26)$$

The convolution is

$$y[n] = (x * h)[n] = \frac{1}{N} \sum_{m=0}^{N-1} x[n-m], \quad (27)$$

subject to NumPy's centered `mode="same"` boundary behavior. Its discrete-time frequency response is

$$H(e^{j\omega}) = \frac{1}{N} e^{-j\omega(N-1)/2} \frac{\sin(N\omega/2)}{\sin(\omega/2)}. \quad (28)$$

Thus it is a rectangular-window low-pass smoother: low-frequency envelope or discriminator trends are preserved while rapid sample-to-sample variations are attenuated.

6.3 Percentile-based robust thresholding

Relevant code (gnuradio_decode_file.py lines 217–244):

```

217 def threshold_binary_robust(x: np.ndarray) -> np.ndarray:
218     vals = x.astype(np.float32)
219     for low_pct, high_pct in ((5, 95), (10, 90), (15, 85), (20, 80), (25, 75)):
220         lo = float(np.percentile(vals, low_pct))
221         hi = float(np.percentile(vals, high_pct))
222         if math.isfinite(lo) and math.isfinite(hi) and hi > lo:
223             threshold = lo + 0.5 * (hi - lo)
224             states = (vals >= threshold).astype(np.uint8)
225             ones = int(np.count_nonzero(states))
226             if 0 < ones < states.size and np.count_nonzero(np.diff(states)) > 0:
227                 return states
228     lo = float(np.min(vals))
229     hi = float(np.max(vals))
230     if math.isfinite(lo) and math.isfinite(hi) and hi > lo:
231         threshold = lo + 0.5 * (hi - lo)
232     return (vals >= threshold).astype(np.uint8)

```

For each percentile pair (p_L, p_H) , the code estimates

$$L = Q_{p_L}(x), \quad H = Q_{p_H}(x), \quad (29)$$

then uses

$$T = \frac{L + H}{2}. \quad (30)$$

The binary state sequence is

$$s[n] = \mathbf{1}\{x[n] \geq T\}. \quad (31)$$

The sequence is accepted only if it contains both states and at least one transition. The search starts with the outer 5th/95th percentiles and progressively moves inward to 25th/75th percentiles. Compared with raw minimum/maximum thresholding, percentiles reduce the influence of isolated extreme samples. If every percentile attempt fails, the code falls back to the ordinary midrange threshold.

6.4 Run-length encoding

Relevant code (gnuradio_decode_file.py lines 194–214):

```

194 def build_runs_from_states(states: np.ndarray) -> list[tuple[int, int]]:
195     runs: list[tuple[int, int]] = []
196     cur = int(states[0])
197     count = 1
198     for val in states[1:]:
199         v = int(val)
200         if v == cur:
201             count += 1
202         else:
203             runs.append((cur, count))
204             cur = v
205             count = 1
206     runs.append((cur, count))
207     return runs

```

A contiguous region with state $a_r \in \{0, 1\}$ and length L_r is represented as the pair

$$R_r = (a_r, L_r). \quad (32)$$

This converts the thresholded sample stream into a pulse-duration representation. For a run spanning sample indices n_a through n_b inclusive,

$$L_r = n_b - n_a + 1. \quad (33)$$

If the true sampling rate is F_s , the physical run duration would be

$$T_r = \frac{L_r}{F_s}, \quad (34)$$

but this helper intentionally works in sample or slot units because it is given `sps`, not F_s .

6.5 Merging short threshold fragments

Relevant code (gnuradio_decode_file.py lines 168–191):

```

168 def merge_short_runs(runs: list[tuple[int, int]], min_len: int) -> list[tuple[int, int]]:
176     out: list[tuple[int, int]] = []
177     for state, length in runs:
178         if length < min_len and out:
179             prev_state, prev_len = out[-1]
180             out[-1] = (prev_state, prev_len + length)
181         else:
182             out.append((state, length))
184     merged: list[tuple[int, int]] = []
185     for state, length in out:
186         if merged and merged[-1][0] == state:
187             prev_state, prev_len = merged[-1]
188             merged[-1] = (prev_state, prev_len + length)
189         else:
190             merged.append((state, length))
191     return merged

```

For any noninitial run satisfying

$$L_r < L_{\min}, \quad (35)$$

the implementation absorbs its length into the preceding run. Adjacent runs that then have the same state are coalesced. In the direct RAW path,

$$L_{\min, \text{merge}} = \max\left(1, \left\lfloor \frac{N_s}{10} \right\rfloor\right). \quad (36)$$

The operation is a heuristic deglitcher: very short state flips are interpreted as threshold fragmentation rather than intentional pulse structure.

6.6 PWM high/low pair decision

Relevant code (gnuradio_decode_file.py lines 247–272):

```

247 def decode_pwm_pairs_from_runs(runs: list[tuple[int, int]], min_len: int) -> list[int]:
254     bits: list[int] = []
256     min_len = max(1, min_len)
258     while i + 1 < len(runs):
259         state_a, len_a = runs[i]
260         state_b, len_b = runs[i + 1]
262         if state_a == 1 and state_b == 0:
264             if len_a >= min_len and len_b >= min_len:
265                 bits.append(1 if len_a > len_b else 0)
266                 i += 2
267                 continue
270             i += 1
272     return bits

```

For each valid high-low run pair

$$(1, L_H), (0, L_L), \quad (37)$$

the recovered bit is

$$b_i = \begin{cases} 1, & L_H > L_L, \\ 0, & L_H \leq L_L. \end{cases} \quad (38)$$

Both durations must satisfy $L_H, L_L \geq L_{\min}$. In the direct sample path,

$$L_{\min} = \max \left(1, \left\lfloor \frac{N_s}{5} \right\rfloor \right). \quad (39)$$

This is a relative-width decoder: it does not need the exact absolute pulse duration as long as the high and low durations remain distinguishable.

6.7 Slot-quantized fallback

Relevant code (gnuradio_decode_file.py lines 307–323):

```

307     if len(bits) >= 2:
308         return bits
310     usable = (env_smooth.size // sps) * sps
311     if usable <= 0:
312         return bits
315     slot_vals = env_smooth[:usable].reshape((-1, sps)).mean(axis=1).astype(np.float32)
316     slot_states = threshold_binary_robust(slot_vals)
317     slot_runs = build_runs_from_states(slot_states)
318     slot_runs = merge_short_runs(slot_runs, 1)
319     slot_bits = decode_pwm_pairs_from_runs(slot_runs, 1)
321     if len(slot_bits) > len(bits):
322         return slot_bits
323     return bits

```

If direct sample-run decoding produces fewer than two bits, the envelope is first averaged into blocks of N_s samples:

$$v_k = \frac{1}{N_s} \sum_{m=0}^{N_s-1} E_s[kN_s + m]. \quad (40)$$

Thresholding and run-length decoding then operate in one-unit slots rather than raw samples. The code returns whichever path produced more recovered bits. This provides a second timing interpretation when individual sample transitions are noisy or slightly misaligned.

7 Complex Phase Discriminator

7.1 Differential phase calculation

Relevant code (gnuradio_decode_file.py lines 326–336):

```

326 def phase_discriminator(samples: np.ndarray) -> np.ndarray:
333     if samples.size < 2:
334         return np.array([], dtype=np.float32)
335     x = normalize_complex(samples)
336     return np.angle(x[1:] * np.conj(x[:-1])).astype(np.float32)

```

Write a complex sample in polar form as

$$x[n] = A[n]e^{j\phi[n]}. \quad (41)$$

Then

$$x[n]x^*[n-1] = A[n]A[n-1]e^{j(\phi[n]-\phi[n-1])}. \quad (42)$$

Taking its argument gives

$$d[n] = \arg\{x[n]x^*[n-1]\} = \text{wrap}_{[-\pi, \pi)}(\phi[n] - \phi[n-1]). \quad (43)$$

Thus the returned values are phase increments in radians per sample. The preliminary unit-power normalization does not alter the phase-difference result because multiplication by a common positive real scale has zero phase.

7.2 Relationship to instantaneous frequency

For a complex sinusoid

$$x[n] = Ae^{j(2\pi fn/F_s + \phi_0)}, \quad (44)$$

its adjacent phase difference is

$$d[n] = \frac{2\pi f}{F_s} \quad (45)$$

when the increment remains inside the principal phase interval. Therefore, if F_s were supplied, frequency in Hz could be recovered by

$$f[n] = \frac{F_s}{2\pi} d[n]. \quad (46)$$

The helper intentionally omits this final scale because its FSK decisions require only relative discriminator levels. It works directly in radians/sample.

8 Symbol Averaging of Real DSP Features

Relevant code (gnuradio_decode_file.py lines 351–362):

```

351 def symbol_average_float(x: np.ndarray, sps: int, max_symbols: int) -> np.ndarray:
352     if x.size == 0 or sps <= 0:
353         return np.array([], dtype=np.float32)
354     usable = (x.size // sps) * sps
355     if usable <= 0:
356         return np.array([], dtype=np.float32)
357     vals = x[:usable].reshape((-1, sps)).mean(axis=1).astype(np.float32)
358     return vals[:max_symbols]
```

For a real-valued feature stream $d[n]$, such as phase-discriminator output, the symbol statistic is

$$v_k = \frac{1}{N_s} \sum_{m=0}^{N_s-1} d[kN_s + m]. \quad (47)$$

This integrates the feature over one assumed symbol period, reducing zero-mean sample noise and producing one decision variable per symbol.

9 2-FSK, GFSK, and AFSK Path

9.1 2-FSK median slicing

Relevant code (gnuradio_decode_file.py lines 365–379):

```

365 def run_fsk2(samples: np.ndarray, sps: int, max_symbols: int, gfsk: bool = False) -> list[int]:
366     freq = phase_discriminator(samples)
367     if gfsk:
368         freq = moving_average(freq, max(3, sps // 3))
369     vals = symbol_average_float(freq, sps, max_symbols)
370     if vals.size == 0:
371         return []
372     threshold = float(np.median(vals))
373     return [1 if v >= threshold else 0 for v in vals]
```

For ordinary 2-FSK, let $d[n]$ be the differential phase. The symbol statistic is

$$v_k = \frac{1}{N_s} \sum_{m=0}^{N_s-1} d[kN_s + m]. \quad (48)$$

The slicing threshold is the sample median of all symbol statistics:

$$T = \text{median}\{v_k\}. \quad (49)$$

The bit decision is

$$b_k = \begin{cases} 1, & v_k \geq T, \\ 0, & v_k < T. \end{cases} \quad (50)$$

For two approximately balanced FSK tones, the median usually lies between the two discriminator clusters. Unlike a fixed zero threshold, this can tolerate a common residual frequency offset that shifts both clusters together.

9.2 Additional GFSK smoothing

When `gfsk=True`, the discriminator is first low-pass smoothed with

$$N_G = \max\left(3, \left\lfloor \frac{N_s}{3} \right\rfloor\right). \quad (51)$$

Thus

$$d_G[n] = \frac{1}{N_G} \sum_{m=0}^{N_G-1} d[n-m]. \quad (52)$$

GFSK intentionally has smoother frequency transitions than rectangular-pulse FSK, so this extra averaging reduces high-frequency discriminator variation before the per-symbol integration stage.

9.3 AFSK dispatch behavior

The dispatch logic sends `afsk` through the same function as 2-FSK, with `gfsk=False`. Therefore the implemented AFSK mathematics in this helper is exactly the same differential-phase, symbol-average, and median-slice chain shown above. There is no separate mark/space matched-filter bank in this source.

10 One-Dimensional K-Means Clustering

10.1 Cluster initialization and assignment

Relevant code (gnuradio_decode_file.py lines 382–406):

```

382 def kmeans_1d(vals: np.ndarray, k: int, rounds: int = 24) -> np.ndarray:
389     if vals.size < k:
390         lo = float(np.min(vals))
391         hi = float(np.max(vals))
392         return np.linspace(lo, hi, k, dtype=np.float32)
394     centers = np.percentile(vals, np.linspace(5, 95, k)).astype(np.float32)
395     for _ in range(rounds):
396         dist = np.abs(vals[:, None] - centers[None, :])
397         labels = np.argmin(dist, axis=1)
398         new_centers = centers.copy()
399         for idx in range(k):
400             part = vals[labels == idx]
401             if part.size:
402                 new_centers[idx] = float(np.mean(part))
403         if np.allclose(new_centers, centers):
404             break
405         centers = new_centers
406     return np.sort(centers)

```

For K clusters with centers c_j , each observation is assigned to its nearest center:

$$\ell_i = \arg \min_{0 \leq j < K} |v_i - c_j|. \quad (53)$$

Each nonempty center is then updated to the mean of its assigned values:

$$c_j^{\text{new}} = \frac{1}{|S_j|} \sum_{v_i \in S_j} v_i. \quad (54)$$

This alternates for at most 24 rounds or until `np.allclose` determines that the centers have effectively stopped moving.

Initialization uses evenly spaced percentiles between 5 and 95:

$$c_j^{(0)} = Q_{p_j}(v), \quad p_j \in \text{linspace}(5, 95, K). \quad (55)$$

Percentile initialization reduces sensitivity to single extreme discriminator samples. If fewer observations than clusters are available, the fallback places K centers evenly between the observed minimum and maximum.

11 4-FSK Demodulation

Relevant code (`gnuradio_decode_file.py` lines 409–421):

```

409 def run_fsk4(samples: np.ndarray, sps: int, max_symbols: int) -> list[int]:
416     vals = symbol_average_float(phase_discriminator(samples), sps, max_symbols)
417     if vals.size == 0:
418         return []
419     centers = kmeans_1d(vals, 4)
420     labels = np.argmax(np.abs(vals[:, None] - centers[None, :]), axis=1)
421     return [int(v) & 0x3 for v in labels]
```

The first two stages are the same differential-phase and per-symbol averaging used for 2-FSK:

$$v_k = \frac{1}{N_s} \sum_{m=0}^{N_s-1} \arg\{x[n]x^*[n-1]\}. \quad (56)$$

Instead of one threshold, the values are clustered into four ordered centers

$$c_0 \leq c_1 \leq c_2 \leq c_3. \quad (57)$$

Each symbol label is

$$s_k = \arg \min_{j \in \{0,1,2,3\}} |v_k - c_j|. \quad (58)$$

Because the centers are sorted before labeling, increasing symbol indices correspond to increasing discriminator level. The final `& 0x3` restricts the integer label to the range 0–3.

12 Ideal PSK and QAM Constellations

12.1 Constellation definitions

Relevant code (`gnuradio_decode_file.py` lines 424–444):

```

424 def constellation_points(mod: str) -> tuple[np.ndarray, int]:
429     if mod == "bpsk":
430         return np.array([-1 + 0j, 1 + 0j], dtype=np.complex64), 1
431     if mod == "qpsk":
432         return np.array([1 + 1j, -1 + 1j, -1 - 1j, 1 - 1j], dtype=np.complex64), 2
433     if mod == "psk8":
434         pts = [np.exp(1j * 2.0 * math.pi * n / 8.0) for n in range(8)]
435         return np.array(pts, dtype=np.complex64), 3
436     if mod == "qam16":
437         levels = [-3, -1, 1, 3]
438         pts = [complex(i, q) for q in levels for i in levels]
439         return np.array(pts, dtype=np.complex64), 4
440     if mod == "qam64":
441         levels = [-7, -5, -3, -1, 1, 3, 5, 7]
442         pts = [complex(i, q) for q in levels for i in levels]
443         return np.array(pts, dtype=np.complex64), 6

```

For BPSK, the ideal set is

$$\mathcal{C}_{\text{BPSK}} = \{-1, +1\}, \quad (59)$$

with one bit per symbol.

For QPSK, the source uses the four points

$$\mathcal{C}_{\text{QPSK}} = \{1 + j, -1 + j, -1 - j, 1 - j\}, \quad (60)$$

with two bits per symbol.

For 8PSK,

$$p_m = e^{j2\pi m/8}, \quad m = 0, \dots, 7, \quad (61)$$

so all eight points lie on the unit circle and carry three bits per symbol.

For square M -QAM, each point is

$$p = I + jQ, \quad (62)$$

with I and Q drawn from equally spaced odd-integer levels. For 16QAM,

$$I, Q \in \{-3, -1, 1, 3\}, \quad (63)$$

and for 64QAM,

$$I, Q \in \{-7, -5, -3, -1, 1, 3, 5, 7\}. \quad (64)$$

The list order defines the fallback integer symbol labels; this source does not apply a separate Gray-code remapping table to these fallback QAM labels.

12.2 Constellation power normalization

Before the NumPy fallback computes distances, the ideal constellation itself is passed through `normalize_complex`. Therefore the reference points have unit average power:

$$\frac{1}{M} \sum_{m=0}^{M-1} |p'_m|^2 = 1. \quad (65)$$

For example, the unnormalized average energy of square 16QAM with levels $\{-3, -1, 1, 3\}$ is

$$E_s = 10, \quad (66)$$

so normalization scales its coordinates by $1/\sqrt{10}$.

13 PSK/QAM Constellation Demodulation

13.1 Optional received-symbol normalization

Relevant code (gnuradio_decode_file.py lines 447–484):

```

447 def run_constellation_demod(symbols: np.ndarray, mod: str, normalize: bool) -> tuple[list[int], int]:
448     if symbols.size == 0:
449         return [], 1
450     x = normalize_complex(symbols) if normalize else symbols.astype(np.complex64)
451     if mod in {"bpsk", "qpsk"}:
452         try:
453             if mod == "bpsk" and hasattr(digital, "constellation_bpsk"):
454                 const = digital.constellation_bpsk().base()
455             elif mod == "qpsk" and hasattr(digital, "constellation_qpsk"):
456                 const = digital.constellation_qpsk().base()
457             tb = gr.top_block()
458             src = blocks.vector_source_c(x.tolist(), False)
459             dec = digital.constellation_decoder_cb(const)
460             sink = blocks.vector_sink_b()
461             tb.connect(src, dec)
462             tb.connect(dec, sink)
463             tb.run()
464             return [int(v) for v in sink.data()], (1 if mod == "bpsk" else 2)
465         except:
466             pts, bps = constellation_points(mod)
467             pts = normalize_complex(pts)
468             labels = np.argmax(np.abs(x[:, None] - pts[None, :]), axis=1)
469             return [int(v) for v in labels], bps

```

When normalization is requested, the received symbol sequence is scaled to unit average power:

$$r'_k = \frac{r_k}{\sqrt{\frac{1}{K} \sum_i |r_i|^2}}. \quad (67)$$

This makes its average scale comparable with the normalized ideal constellation. It does not estimate or remove carrier phase rotation, residual frequency rotation, or symbol-timing error; those conditions must already be sufficiently controlled for nearest-point decoding to work.

13.2 GNU Radio BPSK/QPSK path

For BPSK and QPSK, the script first attempts to use GNU Radio's built-in constellation objects and `constellation_decoder_cb`. Conceptually, a constellation decoder assigns each received symbol to a constellation decision region and outputs the associated symbol index. If the convenience objects are unavailable or the block raises an exception, the code moves to the generic NumPy nearest-point path.

13.3 Euclidean nearest-neighbor fallback

For every received complex symbol r_k and every normalized ideal point p_j , the code calculates the Euclidean complex-plane distance

$$d_{k,j} = |r_k - p_j|. \quad (68)$$

Equivalently,

$$d_{k,j}^2 = (\Re\{r_k - p_j\})^2 + (\Im\{r_k - p_j\})^2. \quad (69)$$

The selected symbol is

$$\hat{s}_k = \arg \min_j d_{k,j}. \quad (70)$$

Because the square root is monotonic, minimizing $d_{k,j}$ is equivalent to minimizing squared Euclidean distance. The NumPy expression uses the magnitude directly.

This is the standard minimum-distance decision rule for equally likely constellation points in isotropic additive white Gaussian noise when the constellation is correctly aligned and scaled.

14 Symbol Label to Bitstream Conversion

14.1 Fixed-width MSB-first extraction

Relevant code (gnuradio_decode_file.py lines 487–509):

```

487 def symbol_to_bits(symbol: int, bps: int, invert: bool) -> str:
488     out = []
489     for bit_index in range(bps - 1, -1, -1):
490         bit = (symbol >> bit_index) & 1
491         if invert:
492             bit ^= 1
493         out.append("1" if bit else "0")
494     return "".join(out)
501 def write_bits(symbols: Iterable[int], bps: int, invert: bool, tight: bool) -> None:
502     sep = "" if tight else " "
503     parts = [symbol_to_bits(int(sym), bps, invert) for sym in symbols]
504     sys.stdout.write(sep.join(parts))
505     sys.stdout.write("\n")

```

For a symbol integer s and B bits per symbol, the bit at position q is extracted by

$$b_q = (s \gg q) \& 1. \quad (71)$$

The loop runs from $q = B - 1$ down to 0, so output is most-significant-bit first. If inversion is enabled,

$$b'_q = b_q \oplus 1 = 1 - b_q. \quad (72)$$

Thus, for example, with $B = 2$,

$$s = 2 = (10)_2 \rightarrow 10, \quad (73)$$

and inversion yields 01.

15 Top-Level Decode Dispatch

Relevant code (gnuradio_decode_file.py lines 537–576):

```

537     try:
538         complex_symbols, mag_symbols, raw_samples = symbol_views(
539             path, args.start_sample, args.max_symbols, args.sps
540         )
541     if args.mod == "ook":
542         symbols = run_ook_symbol_slicer(mag_symbols, bool(args.normalize))
543         natural_bps = 1
544     elif args.mod == "ook_raw":
545         symbols = run_ook_raw_pulse_decoder(raw_samples, args.sps, bool(args.normalize))
546         natural_bps = 1
547     elif args.mod in {"fsk2", "gfsk", "afsk"}:
548         symbols = run_fsk2(raw_samples, args.sps, args.max_symbols, gfsk=(args.mod == "gfsk"))
549         natural_bps = 1
550     elif args.mod == "fsk4":
551         symbols = run_fsk4(raw_samples, args.sps, args.max_symbols)
552         natural_bps = 2
553     elif args.mod in {"bpsk", "qpsk", "psk8", "qam16", "qam64"}:
554         symbols, natural_bps = run_constellation_demod(complex_symbols, args.mod, bool(args.normalize))
555     bps = user_bps if 1 <= user_bps <= 8 else natural_bps
556     write_bits(symbols, bps, invert, tight)

```

The dispatch structure shows which intermediate representation feeds each modulation family:

Mode	Input statistic	Decision mechanism
OOK	mean symbol power P_k	midrange threshold through GNU Radio
OOK RAW	raw envelope power $E[n]$	robust threshold + run-length PWM comparison
2-FSK / AFSK	differential phase $d[n]$	symbol mean + median threshold
GFSK	smoothed differential phase	symbol mean + median threshold
4-FSK	differential phase	symbol mean + 4-center k-means
BPSK / QPSK	complex symbol average $\bar{x}_k$	GNU Radio decoder when available; otherwise nearest point
8PSK / 16QAM / 64QAM	complex symbol average $\bar{x}_k$	Euclidean nearest ideal point

The output bit width is selected by

$$B = \begin{cases} B_{\text{user}}, & 1 \leq B_{\text{user}} \leq 8, \\ B_{\text{natural}}, & \text{otherwise.} \end{cases} \quad (74)$$

This means the explicit command-line bits-per-symbol value can override the natural modulation width.

16 End-to-End Mathematical Paths

16.1 OOK symbol slicer

Starting from complex16 storage,

$$x[n] = \frac{I_{16}[n] + jQ_{16}[n]}{32768}, \quad (75)$$

$$P_k = \frac{1}{N_s} \sum_{m=0}^{N_s-1} |x[kN_s + m]|^2, \quad (76)$$

$$T = \frac{\min(P) + \max(P)}{2}, \quad (77)$$

$$b_k = \mathbf{1}\{P_k \geq T\}. \quad (78)$$

If normalization is enabled, P_k is first transformed to the $[0, 1]$ interval.

16.2 2-FSK / GFSK / AFSK

The core path is

$$x[n] = A[n]e^{j\phi[n]}, \quad (79)$$

$$d[n] = \arg\{x[n]x^*[n-1]\}, \quad (80)$$

$$\tilde{d}[n] = d[n] * h[n] \quad (\text{GFSK only}), \quad (81)$$

$$v_k = \frac{1}{N_s} \sum_{m=0}^{N_s-1} \tilde{d}[kN_s + m], \quad (82)$$

$$T = \text{median}\{v_k\}, \quad (83)$$

$$b_k = \mathbf{1}\{v_k \geq T\}. \quad (84)$$

For ordinary FSK2 and AFSK, $\tilde{d} = d$.

16.3 4-FSK

The first three calculations are the same through v_k . Then

$$\{c_0, c_1, c_2, c_3\} = \text{kmeans}_4(\{v_k\}), \quad (85)$$

$$s_k = \arg \min_j |v_k - c_j|. \quad (86)$$

The resulting $s_k \in \{0, 1, 2, 3\}$ is normally represented by two output bits.

16.4 PSK/QAM

For fixed symbol timing,

$$\bar{x}_k = \frac{1}{N_s} \sum_{m=0}^{N_s-1} x[kN_s + m], \quad (87)$$

$$r_k = \frac{\bar{x}_k}{\sqrt{\frac{1}{K} \sum_i |\bar{x}_i|^2}} \quad (\text{if normalize}), \quad (88)$$

$$p'_j = \frac{p_j}{\sqrt{\frac{1}{M} \sum_m |p_m|^2}}, \quad (89)$$

$$\hat{s}_k = \arg \min_j |r_k - p'_j|. \quad (90)$$

For BPSK/QPSK, GNU Radio may supply the decision stage instead of the explicit final nearest-point equation.

17 Worked DSP Examples

17.1 OOK symbol-power example

Suppose four normalized symbol-power values are

$$P = \{0.08, 0.92, 0.11, 0.88\}. \quad (91)$$

Then

$$T = 0.08 + \frac{1}{2}(0.92 - 0.08) = 0.50. \quad (92)$$

The decisions are therefore

$$\{0, 1, 0, 1\}. \quad (93)$$

17.2 RAW PWM example

Assume the thresholded envelope produces the run pairs

$$(1, 30), (0, 10), (1, 10), (0, 30). \quad (94)$$

For the first pair, $30 > 10$, so

$$b_0 = 1. \quad (95)$$

For the second pair, $10 \leq 30$, so

$$b_1 = 0. \quad (96)$$

The recovered bit sequence is 10.

17.3 2-FSK phase-discriminator example

Suppose the per-symbol averaged discriminator levels are

$$v = \{-0.24, 0.27, -0.22, 0.25\} \text{ rad/sample.} \quad (97)$$

The median lies near

$$T \approx 0.015 \text{ rad/sample.} \quad (98)$$

Therefore the decisions are

$$\{0, 1, 0, 1\}. \quad (99)$$

If the sample rate were 2 MS/s, a discriminator value of 0.25 rad/sample would correspond to the frequency offset

$$f = \frac{2 \times 10^6}{2\pi}(0.25) \approx 79.6 \text{ kHz.} \quad (100)$$

The helper does not need to perform this Hz conversion to distinguish the two tones.

17.4 QPSK nearest-point example

After normalization, suppose a received symbol is

$$r = 0.68 + 0.73j. \quad (101)$$

The normalized QPSK point in the first quadrant is

$$p_0 = \frac{1+j}{\sqrt{2}} \approx 0.707 + 0.707j. \quad (102)$$

Its distance is

$$d_0 = |r - p_0| \quad (103)$$

$$\approx \sqrt{(0.68 - 0.707)^2 + (0.73 - 0.707)^2} \quad (104)$$

$$\approx 0.035. \quad (105)$$

If that is the minimum distance among all four ideal points, the symbol receives the corresponding constellation label.

18 Engineering Interpretation and Assumptions

18.1 Fixed timing is an input assumption

The script assumes the supplied `sps` correctly identifies the symbol period and that the requested starting sample is acceptably aligned. Its averaging functions use nonoverlapping blocks beginning exactly at that offset. Therefore timing error changes which samples enter each symbol average.

18.2 No explicit carrier recovery in the constellation fallback

The PSK/QAM fallback performs amplitude normalization but does not estimate a static phase rotation or residual carrier-frequency rotation. If

$$r_k = s_k e^{j\theta}, \quad (106)$$

with significant unknown θ , normalization alone cannot remove θ . This helper consequently expects the input constellation to be sufficiently aligned for the chosen decision points.

18.3 Differential phase naturally suppresses amplitude scale

Because

$$\arg\{\alpha x[n](\alpha x[n-1])^*\} = \arg\{|\alpha|^2 x[n]x^*[n-1]\}, \quad (107)$$

any positive common gain factor disappears from the phase result. This is why the FSK discriminator fundamentally depends on phase progression rather than received amplitude.

18.4 Median FSK threshold adapts to common offset

If every FSK decision statistic receives a common offset c ,

$$v'_k = v_k + c, \quad (108)$$

then

$$\text{median}(v') = \text{median}(v) + c. \quad (109)$$

Therefore the binary comparisons are unchanged as long as the two tone clusters remain separable. This gives the implemented 2-FSK slicer tolerance to a common discriminator bias.

18.5 K-means estimates tone levels from the recording

The 4-FSK path does not require four predetermined phase-increment values. It estimates the four centers from the observed symbol statistics. This allows a common frequency offset and unknown deviation scale to move the cluster locations without requiring hard-coded frequencies, although the cluster populations still need to be distinguishable.

18.6 Constellation labeling is implementation-defined by point order

For the NumPy fallback, `argmin` returns the index of the nearest point in the arrays created by `constellation_points`. That index is subsequently converted directly to bits. The resulting label order is therefore the source array order; it is not a separate protocol-specific Gray mapping or differential decoder.

19 DSP Function Reference Summary

Function	Primary mathematics	Role
<code>read_complex16_raw</code>	$(I + jQ)/32768$	Load and normalize interleaved complex16 IQ
<code>symbol_views</code>	block means; mean $ x ^2$	Build complex and power symbol statistics
<code>normalize_complex</code>	$x/\sqrt{E x ^2}$	Unit-average-power normalization
<code>normalize_float</code>	$(x - x_{\min})/(x_{\max} - x_{\min})$	Scale real data to $[0, 1]$
<code>run_ook_symbol_slicer</code>	midpoint threshold	OOK/ASK symbol decisions
<code>threshold_binary_robust</code>	percentile midpoint	Robust RAW envelope state detection
<code>build_runs_from_states</code>	run-length encoding	Convert states to pulse durations
<code>merge_short_runs</code>	duration threshold	Suppress short threshold fragments
<code>decode_pwm_pairs_from_runs</code>	$L_H > L_L$	Decode relative-width RAW OOK pulses

Function	Primary mathematics	Role
<code>run_ook_raw_pulse_decoder</code>	envelope + smoothing + RLE	Complete RAW OOK decode chain
<code>phase_discriminator</code>	$\arg(x[n]x^*[n-1])$	Instantaneous phase increment
<code>moving_average</code>	rectangular FIR convolution	Smooth envelope/discriminator data
<code>symbol_average_float</code>	block mean	Integrate one real statistic per symbol
<code>run_fsk2</code>	median slice of phase increments	2-FSK/GFSK/AFSK decisions
<code>kmeans_1d</code>	nearest-center assignment + centroid mean	Estimate discrete frequency levels
<code>run_fsk4</code>	4-center nearest decision	4-FSK symbol labels
<code>constellation_points</code>	ideal PSK/QAM geometry	Build reference constellations
<code>run_constellation_demod</code>	$\arg \min_j r - p_j $	PSK/QAM symbol decisions
<code>symbol_to_bits</code>	shifts, masks, XOR	Convert symbol labels to bits

20 Core DSP Source Listings

This section groups the principal DSP calculations in source order for quick reference. The line numbers are the actual line numbers in the supplied `gnuradio_decode_file.py`.

20.1 Complex16 normalization and symbol statistics

```

64  with path.open("rb") as fp:
65      fp.seek(start_sample * 4, 0)
66      raw = np.fromfile(fp, dtype=np.int16, count=count_iq * 2)
74  iq = raw.reshape((-1, 2)).astype(np.float32)
76  return ((iq[:, 0] + 1j * iq[:, 1]) / 32768.0).astype(np.complex64)
91  c = read_complex16_raw(path, start_sample, max_symbols * sps)
92  usable = (c.size // sps) * sps
101 c = c[:usable]
102 blocks_view = c.reshape((-1, sps))
103 complex_symbols = blocks_view.mean(axis=1).astype(np.complex64)
104 mag_symbols = np.mean(np.abs(blocks_view) ** 2, axis=1).astype(np.float32)

```

20.2 Normalization

```

116 p = float(np.mean(np.abs(x) ** 2))
117 if p <= 0.0 or not math.isfinite(p):
118     return x.astype(np.complex64)
119 return (x / math.sqrt(p)).astype(np.complex64)
129 lo = float(np.min(x))
130 hi = float(np.max(x))
131 if not math.isfinite(lo) or not math.isfinite(hi) or hi <= lo:
132     return x.astype(np.float32)
133 return ((x - lo) / (hi - lo)).astype(np.float32)

```

20.3 RAW OOK DSP

```

227     for low_pct, high_pct in ((5, 95), (10, 90), (15, 85), (20, 80), (25, 75)):
228         lo = float(np.percentile(vals, low_pct))
229         hi = float(np.percentile(vals, high_pct))
231         threshold = lo + 0.5 * (hi - lo)
232         states = (vals >= threshold).astype(np.uint8)
262         if state_a == 1 and state_b == 0:
264             if len_a >= min_len and len_b >= min_len:
265                 bits.append(1 if len_a > len_b else 0)
297     env = (np.abs(samples) ** 2).astype(np.float32)
298     smooth_len = max(1, min(max(1, sps // 20), 128))
299     env_smooth = moving_average(env, smooth_len)
303     states = threshold_binary_robust(env_smooth)
304     runs = build_runs_from_states(states)
305     runs = merge_short_runs(runs, max(1, sps // 10))
306     bits = decode_pwm_pairs_from_runs(runs, max(1, sps // 5))

```

20.4 FSK-family DSP

```

335     x = normalize_complex(samples)
336     return np.angle(x[1:] * np.conj(x[:-1])).astype(np.float32)
347     kernel = np.ones(n, dtype=np.float32) / float(n)
348     return np.convolve(x, kernel, mode="same").astype(np.float32)
361     vals = x[:usable].reshape((-1, sps)).mean(axis=1).astype(np.float32)
372     freq = phase_discriminator(samples)
373     if gfsk:
374         freq = moving_average(freq, max(3, sps // 3))
375     vals = symbol_average_float(freq, sps, max_symbols)
378     threshold = float(np.median(vals))
379     return [1 if v >= threshold else 0 for v in vals]

```

20.5 4-FSK clustering

```

394     centers = np.percentile(vals, np.linspace(5, 95, k)).astype(np.float32)
395     for _ in range(rounds):
396         dist = np.abs(vals[:, None] - centers[None, :])
397         labels = np.argmin(dist, axis=1)
400         part = vals[labels == idx]
401         if part.size:
402             new_centers[idx] = float(np.mean(part))
406     return np.sort(centers)
416     vals = symbol_average_float(phase_discriminator(samples), sps, max_symbols)
419     centers = kmeans_1d(vals, 4)
420     labels = np.argmin(np.abs(vals[:, None] - centers[None, :]), axis=1)
421     return [int(v) & 0x3 for v in labels]

```

20.6 PSK/QAM decisions

```

429     if mod == "bpsk":
430         return np.array([-1 + 0j, 1 + 0j], dtype=np.complex64), 1
431     if mod == "qpsk":
432         return np.array([1 + 1j, -1 + 1j, -1 - 1j, 1 - 1j], dtype=np.complex64), 2
433     if mod == "psk8":
434         pts = [np.exp(1j * 2.0 * math.pi * n / 8.0) for n in range(8)]
436     if mod == "qam16":
437         levels = [-3, -1, 1, 3]
438         pts = [complex(i, q) for q in levels for i in levels]
440     if mod == "qam64":
441         levels = [-7, -5, -3, -1, 1, 3, 5, 7]
442         pts = [complex(i, q) for q in levels for i in levels]
457     x = normalize_complex(samples) if normalize else samples.astype(np.complex64)
481     pts, bps = constellation_points(mod)
482     pts = normalize_complex(pts)
483     labels = np.argmin(np.abs(x[:, None] - pts[None, :]), axis=1)

```